Raport — Lucrarea de laborator nr. 6

Binary Search Trees, DFS și BFS

Probleme LeetCode: 701, 94, 102 · Limbaj: Python 3

Limbaj: **Python 3**. Nodul folosește câmpurile LeetCode: `val`, `left`, `right`; copilul gol este `None`.

Fișier	Conținut
`lab6.py`	`TreeNode`, inserare iterativă + recursivă, `build_bst`, inorder DFS, BFS pe niveluri, funcții de trasare
`test_lab6.py`	toate testele locale (rulare: `python3 test_lab6.py`)
`test_output.txt`	ieșirea completă a testelor (33/33 trecute)
`leetcode/701_insert_into_bst.py`, `leetcode/094_inorder_traversal.py`, `leetcode/102_level_order.py`	codul de trimis pe LeetCode

1. Construirea unui BST (LeetCode 701)

Algoritm

Pornim de la rădăcină și comparăm: `val < nod.val` → stânga, `val > nod.val` → dreapta, până găsim o legătură goală; acolo legăm o **frunză nouă**. Nodurile existente nu se mută. Arbore gol → nodul nou devine rădăcina. Duplicat (versiunea locală) → arborele rămâne neschimbat. Varianta iterativă este cea principală (fără risc de depășire a stivei); varianta recursivă salvează rezultatul în `root.left = ...` / `root.right = ...`.

Desenul arborelui după fiecare inserare și drumul de comparații

Intrare: `[4, 2, 7, 1, 3, 6, 9]`, pornind de la `root = None`, cu `root = insert(root, v)` după fiecare pas.

Pas	Valoare	Drum de comparații	Rezultat
1	4	arbore gol	4 devine rădăcină
2	2	2<4 → stânga	copil stâng al lui 4
3	7	7>4 → dreapta	copil drept al lui 4
4	1	1<4 → stânga, 1<2 → stânga	copil stâng al lui 2
5	3	3<4 → stânga, 3>2 → dreapta	copil drept al lui 2
6	6	6>4 → dreapta, 6<7 → stânga	copil stâng al lui 7
7	9	9>4 → dreapta, 9>7 → dreapta	copil drept al lui 7

```
după 4:   4

după 2:     4
           /
          2

după 7:     4
           / \
          2   7

după 1:       4
             / \
            2   7
           /
          1

după 3:       4
             / \
            2   7
           / \
          1   3

după 6:       4
             / \
            2   7
           / \ /
          1  3 6

după 9:       4          (arborele așteptat)
             / \
            2   7
           / \ / \
          1  3 6  9
```

Predicția pentru 5 (pe un arbore nou construit din aceeași listă)

Drum: 5>4 → dreapta, 5<7 → stânga, 5<6 → stânga → **5 devine copilul stâng al lui 6**. Confirmat de test.

```
       4
      / \
     2   7
    / \ / \
   1  3 6  9
      /
     5
```

Teste locale de inserare (toate trecute)

Test	Rezultat
intrare goală `[]`	`None`
o valoare `[10]`	nod 10 fără copii
valori negative `[0,-5,5,-10,-1]`	inorder `[-10,-5,-1,0,5]`, niveluri `[[0],[-5,5],[-10,-1]]`
duplicat: inserarea lui 3 în arborele de bază	arbore identic (aceleași noduri și legături)
duplicate `[5,3,5,3,8,8]`	inorder `[3,5,8]`
intrare sortată `[1,2,3,4]`	**lanț doar spre dreapta**, înălțime 3
intrare sortată descrescător `[4,3,2,1]`	lanț doar spre stânga
inserarea lui 8	nodurile existente păstrează poziția; 8 devine copilul stâng al lui 9
recursiv vs. iterativ	același arbore
lanț de 5000 de noduri (iterativ)	înălțime 4999, fără depășirea stivei

```
[1, 2, 3, 4] →  1
                 \
                  2
                   \
                    3
                     \
                      4
```

Explicații

- **De ce frunza nouă păstrează regula pentru tot subarborele?** La fiecare strămoș `A` de pe drum am mers la stânga doar dacă `val < A.val` și la dreapta doar dacă `val > A.val`. Deci noua valoare ajunge în subarborele stâng al fiecărui strămoș mai mare decât ea și în subarborele drept al fiecărui strămoș mai mic. Pentru nodurile din afara drumului nu se schimbă nimic, iar frunza nouă nu are subarbori. Regula BST rămâne adevărată pentru fiecare nod.
- **De ce trebuie apelantul să salveze rădăcina returnată?** Când arborele este gol, inserarea creează un nod nou

care *este* rădăcina; fără `root = insert(root, v)` acest nod s-ar pierde și arborele ar rămâne `None`. În varianta recursivă, la fiecare nivel `root.left = insert(root.left, v)` leagă noua frunză de părinte.

- **De ce o inserare costă `O(h + 1)`?** Facem o comparație pe fiecare nod de pe un singur drum de la rădăcină în jos; un drum are cel mult `h + 1` noduri (`h` muchii). `+1` acoperă arborele cu un singur nod sau gol (h = 0 / -1), unde tot se face un pas constant.
- **De ce construirea cu `n` valori distincte poate costa `O(n^2)`?** Cu intrare sortată fiecare valoare nouă merge la capătul unui lanț: a `k`-a inserare parcurge `k − 1` noduri. Total `0 + 1 + … + (n − 1) = n(n − 1)/2 = O(n^2)`.
- **De ce `O(n log n)` dacă arborele rămâne echilibrat?** Înălțimea unui arbore echilibrat cu `k` noduri este `O(log k)`, deci fiecare dintre cele `n` inserări costă `O(log n)`; total `O(n log n)`. Un BST obișnuit **nu garantează** echilibrarea — depinde de ordinea de inserare.

Costuri

	Timp	Spațiu de lucru	Stocare
o inserare iterativă	`O(h + 1)`	`O(1)`	+1 nod
o inserare recursivă	`O(h + 1)`	`O(h + 1)` stivă de apeluri	+1 nod
construire `n` valori	`O(n^2)` cel mai rău, `O(n log n)` echilibrat	`O(1)` (iterativ)	`O(n)` noduri

---

2. DFS: parcurgerea inorder (LeetCode 94)

Algoritm

`dfs(nod)`: dacă `nod` e `None` → return; altfel `dfs(stânga)`, adaugă `nod.val`, `dfs(dreapta)`. Rezultatul este **o singură listă partajată** — nu se copiază liste la întoarcerea din apeluri. Parcurgerea urmează legăturile; valorile nu se sortează.

Traseul apelurilor pe arborele din Problema 1

Fiecare apel: când începe (`start`), când adaugă valoarea (`add`), când revine (`return`).

```
start 4
  start 2
    start 1
      call(None) -> return
    add 1   result=[1]
      call(None) -> return
    return 1
  add 2   result=[1, 2]
    start 3
      call(None) -> return
    add 3   result=[1, 2, 3]
      call(None) -> return
    return 3
  return 2
add 4   result=[1, 2, 3, 4]
  start 7
    start 6
      call(None) -> return
    add 6   result=[1, 2, 3, 4, 6]
      call(None) -> return
    return 6
  add 7   result=[1, 2, 3, 4, 6, 7]
    start 9
      call(None) -> return
    add 9   result=[1, 2, 3, 4, 6, 7, 9]
      call(None) -> return
    return 9
  return 7
return 4
```

Rezultat: `[1, 2, 3, 4, 6, 7, 9]`.

Primul apel își adaugă valoarea primul? Nu. Apelul pentru rădăcina 4 începe primul, dar adaugă 4 abia al patrulea, după ce tot subarborele stâng (1, 2, 3) a fost terminat. Prima valoare adăugată este 1, cel mai din stânga nod.

Teste (toate trecute)

Test	Rezultat
BST din Problema 1	`[1, 2, 3, 4, 6, 7, 9]`
arborele din curs care **nu** e BST (2; 1, 3; 4 sub 1)	`[4, 1, 2, 3]`
arbore gol	`[]`
un nod	`[7]`
lanț doar stâng 3→2→1	`[1, 2, 3]`
lanț doar drept 1→2→3	`[1, 2, 3]`
arbore neschimbat după parcurgere	da (comparăm identitatea nodurilor, valorile și legăturile înainte/după)

Explicații

- **De ce fiecare nod e adăugat exact o dată?** Arborele este valid (fără cicluri, fiecare nod are un singur părinte), deci fiecare nod este atins pe exact un drum de la rădăcină → `dfs` este apelat o singură dată pentru el, iar acel apel face exact un `append`.
- **De ce se termină recursia?** Fiecare apel recursiv coboară un nivel, la un copil. Într-un arbore finit fără cicluri adâncimea este cel mult `h`, deci orice lanț de apeluri ajunge la `None` (cazul de bază) și revine. Numărul total de apeluri este finit (`n` noduri + `n + 1` copii goi).
- **De ce inorder este sortat pentru un BST, dar nu pentru orice arbore binar?** Inorder scrie tot subarborele stâng, apoi nodul, apoi tot subarborele drept. Într-un BST stânga < nod < dreapta, iar prin inducție fiecare subarbore iese sortat, deci concatenarea e sortată. Într-un arbore oarecare nu există această relație: în exemplul din curs 4 este în stânga lui 1, deci apare înaintea lui → `[4, 1, 2, 3]`.
- **De ce timpul este `O(n)`, chiar și pentru un BST echilibrat?** Parcurgerea trebuie să viziteze **toate** cele `n` noduri (plus `n + 1` copii goi), cu muncă `O(1)` la fiecare. Echilibrarea scurtează doar drumul spre *un* nod (căutarea), nu numărul de noduri care trebuie vizitate.

Costuri

Timp	Spațiu de lucru	Ieșire
`O(n)`	`O(h + 1)` stiva de apeluri (`O(log n)` echilibrat, `O(n)` lanț)	`O(n)` lista rezultat

Un lanț foarte adânc poate depăși limita de recursie din Python (~1000), de aceea testele recursive locale sunt mici.

---

3. BFS: o listă pe nivel (LeetCode 102)

Algoritm

Arbore gol → `[]`. Coada (`collections.deque`, `append`/`popleft` în `O(1)`) pornește cu rădăcina. La începutul fiecărui nivel salvăm `level_size = len(queue)`, scoatem exact atâtea noduri, le adăugăm valorile în lista nivelului și punem în coadă copilul stâng, apoi pe cel drept. Coada stochează **referințe la noduri**, nu valori.

Traseul cozii pe arborele din Problema 1

Coada la început de nivel	Noduri de scos	Rezultat nivel
`[4]`	1	`[4]`
`[2, 7]`	2	`[2, 7]`
`[1, 3, 6, 9]`	4	`[1, 3, 6, 9]`

Coada după fiecare scoatere și adăugarea copiilor:

```
Început nivel: coada=[4], de scos=1
  scos 4 -> coada=[2, 7]
  nivel=[4]
Început nivel: coada=[2, 7], de scos=2
  scos 2 -> coada=[7, 1, 3]
  scos 7 -> coada=[1, 3, 6, 9]
  nivel=[2, 7]
Început nivel: coada=[1, 3, 6, 9], de scos=4
  scos 1 -> coada=[3, 6, 9]
  scos 3 -> coada=[6, 9]
  scos 6 -> coada=[9]
  scos 9 -> coada=[]
  nivel=[1, 3, 6, 9]
```

Rezultat final: `[[4], [2, 7], [1, 3, 6, 9]]`.

Observație: după scoaterea lui 2, coada `[7, 1, 3]` conține părți din **două niveluri vecine** (1 și 2).

Teste (toate trecute)

Test	Rezultat
BST din Problema 1	`[[4], [2, 7], [1, 3, 6, 9]]`
arbore gol	`[]`
un nod	`[[7]]`
lanț drept `[1,2,3,4]`	`[[1], [2], [3], [4]]`
lanț stâng 3→2→1	`[[3], [2], [1]]`
arborele din Problema 2	`[[2], [1, 3], [4]]`
arbore neschimbat după parcurgere	da

Explicații

- **De ce salvăm dimensiunea nivelului înainte de a adăuga copiii?** În timp ce scoatem noduri, copiii lor intră în aceeași coadă. Dacă am folosi `len(queue)` din mers sau am scoate până se golește coada, copiii (nivelul următor) ar ajunge în lista nivelului curent. Numărul salvat la început delimitează exact nodurile nivelului curent.
- **De ce adăugarea copilului stâng primul dă ordinea cerută?** Coada este FIFO. Nodurile unui nivel sunt scoase de la stânga la dreapta; fiecare își pune copiii stâng-apoi-drept. Prin inducție, nivelul următor intră în coadă tot de la stânga la dreapta, deci este scos în aceeași ordine.
- **De ce fiecare nod intră și iese din coadă exact o dată?** Rădăcina intră o dată la început. Orice alt nod intră doar când este scos părintele lui; are un singur părinte, iar părintele e scos o singură dată, deci nodul intră o dată. Fiecare nod introdus este scos o dată (bucla continuă până se golește coada).
- **De ce timpul este `O(n)`?** `n` operații de intrare și `n` de ieșire, fiecare `O(1)` la `deque`, plus `O(1)` muncă per nod (adăugarea valorii, verificarea a doi copii). Total `O(n)`. O listă Python cu `pop(0)` ar deplasa elementele și ar da `O(n^2)` — de aceea folosim `deque`.

Costuri

Timp	Spațiu de lucru	Ieșire
`O(n)`	`O(w)` coada (doar nodurile în așteptare; `w` = nivelul cel mai lat; poate conține părți din două niveluri vecine)	`O(n)` listele returnate

Pentru arborele de bază `w = 4`. Coada folosită (`deque`) nu alocă un tablou pentru toate cele `n` noduri.

Rezumat costuri

Operație	Timp	Spațiu de lucru	Stocare / ieșire
Inserare (iterativă)	`O(h + 1)`	`O(1)`	+1 nod
Inserare (recursivă)	`O(h + 1)`	`O(h + 1)` stivă	+1 nod
Construire BST din `n` valori	`O(n^2)` cel mai rău / `O(n log n)` echilibrat	`O(1)`	`O(n)` noduri
Inorder DFS recursiv	`O(n)`	`O(h + 1)` stivă	`O(n)`
BFS pe niveluri	`O(n)`	`O(w)` coadă	`O(n)`

Memoria: Python gestionează automat memoria (garbage collector), deci nu este necesară eliberarea manuală.

Verificare finală

- `python3 test_lab6.py` → **Toate cele 33 teste au trecut** (ieșirea completă în `test_output.txt`).
- Soluțiile din `leetcode/` au fost verificate local pe aceleași arbori; trimite-le pe LeetCode (Python3) și adaugă capturile **Accepted** pentru 701, 94 și 102.

Anexa A. Soluțiile trimise pe LeetCode

701. Insert into a Binary Search Tree

```python
# 701. Insert into a Binary Search Tree  (Python3)
# Timp O(h + 1), spațiu extra O(1) (iterativ).
#
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def insertIntoBST(self, root: Optional[TreeNode], val: int) -> Optional[TreeNode]:
        new_node = TreeNode(val)
        if root is None:
            return new_node
        cur = root
        while True:
            if val < cur.val:
                if cur.left is None:
                    cur.left = new_node
                    return root
                cur = cur.left
            elif val > cur.val:
                if cur.right is None:
                    cur.right = new_node
                    return root
                cur = cur.right
            else:               # duplicat (LeetCode nu îl trimite)
                return root
```

94. Binary Tree Inorder Traversal

```python
# 94. Binary Tree Inorder Traversal  (Python3)
# DFS recursiv, o singură listă partajată. Timp O(n), stivă O(h + 1), ieșire O(n).
#
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        result = []

        def dfs(node):
            if node is None:
                return
            dfs(node.left)
            result.append(node.val)
            dfs(node.right)

        dfs(root)
        return result
```

102. Binary Tree Level Order Traversal

```python
# 102. Binary Tree Level Order Traversal  (Python3)
# BFS cu deque (O(1) append/popleft). Timp O(n), coadă O(w), ieșire O(n).
#
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from collections import deque

class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        if root is None:
            return []
        result = []
        queue = deque([root])
        while queue:
            level_size = len(queue)
            level = []
            for _ in range(level_size):
                node = queue.popleft()
                level.append(node.val)
                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)
            result.append(level)
        return result
```

Anexa B. Codul local (`lab6.py`)

```python
"""
Lab 6: Binary Search Trees, DFS, and BFS  (Python 3)

Conține:
  - TreeNode (aceleași câmpuri ca la LeetCode: val, left, right)
  - insert_into_bst  : inserare iterativă, O(h + 1) timp, O(1) spațiu extra
  - insert_into_bst_recursive : variantă recursivă (salvează rădăcina returnată)
  - build_bst        : construiește un BST pornind de la o rădăcină goală
  - inorder_traversal: DFS recursiv inorder, cu o singură listă partajată
  - level_order      : BFS cu coadă FIFO (collections.deque, O(1) enqueue/dequeue)
  - funcții de trasare (drum de comparații, apeluri DFS, coadă BFS)
"""

from collections import deque
from typing import List, Optional


# ------------------------------------------------------------------------
# Nod
# ------------------------------------------------------------------------
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ------------------------------------------------------------------------
# Problema 1: inserare în BST (LeetCode 701)
# ------------------------------------------------------------------------
def insert_into_bst(root: Optional[TreeNode], val: int) -> Optional[TreeNode]:
    """Inserare iterativă. Returnează rădăcina (eventual nouă).
    Un duplicat lasă arborele neschimbat (regula din curs)."""
    new_node = TreeNode(val)
    if root is None:                # arbore gol -> nodul nou devine rădăcină
        return new_node

    cur = root
    while True:
        if val < cur.val:
            if cur.left is None:    # poziție goală găsită -> frunză nouă
                cur.left = new_node
                return root
            cur = cur.left
        elif val > cur.val:
            if cur.right is None:
                cur.right = new_node
                return root
            cur = cur.right
        else:                       # duplicat -> nicio modificare
            return root


def insert_into_bst_recursive(root: Optional[TreeNode], val: int) -> Optional[TreeNode]:
    """Variantă recursivă: rădăcina subarborelui returnat se salvează
    în legătura corectă (root.left / root.right)."""
    if root is None:
        return TreeNode(val)
    if val < root.val:
        root.left = insert_into_bst_recursive(root.left, val)
    elif val > root.val:
        root.right = insert_into_bst_recursive(root.right, val)
    return root


def build_bst(values: List[int]) -> Optional[TreeNode]:
    """Pornește de la o rădăcină goală și inserează valorile în ordine,
    salvând rădăcina returnată după fiecare inserare."""
    root = None
    for v in values:
        root = insert_into_bst(root, v)
    return root


# ------------------------------------------------------------------------
# Problema 2: DFS inorder (LeetCode 94)
# ------------------------------------------------------------------------
def inorder_traversal(root: Optional[TreeNode]) -> List[int]:
    result: List[int] = []          # o singură listă partajată

    def dfs(node: Optional[TreeNode]) -> None:
        if node is None:            # copil gol -> stop
            return
        dfs(node.left)              # 1. subarborele stâng
        result.append(node.val)     # 2. nodul curent
        dfs(node.right)             # 3. subarborele drept

    dfs(root)
    return result
```

```python
# ---------------------------------------------------------------------------
# Problema 3: BFS pe niveluri (LeetCode 102)
# ---------------------------------------------------------------------------
def level_order(root: Optional[TreeNode]) -> List[List[int]]:
    if root is None:
        return []
    result: List[List[int]] = []
    queue = deque([root])            # coadă de referințe la noduri
    while queue:
        level_size = len(queue)      # salvat ÎNAINTE de a adăuga copiii
        level: List[int] = []
        for _ in range(level_size):
            node = queue.popleft()   # O(1)
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result


# ---------------------------------------------------------------------------
# Utilitare pentru teste și trasee
# ---------------------------------------------------------------------------
def height(root: Optional[TreeNode]) -> int:
    """Înălțimea în muchii; arborele gol are -1, un singur nod are 0."""
    # iterativ (pe niveluri), ca să funcționeze și pe lanțuri foarte lungi
    h = -1
    level = [root] if root else []
    while level:
        h += 1
        level = [c for n in level for c in (n.left, n.right) if c]
    return h


def snapshot(root: Optional[TreeNode]):
    """Structura completă (identitatea nodurilor + valori + legături),
    folosită pentru a verifica faptul că parcurgerea nu modifică arborele."""
    if root is None:
        return None
    return (id(root), root.val, snapshot(root.left), snapshot(root.right))


def is_right_chain(root: Optional[TreeNode]) -> bool:
    while root:
        if root.left is not None:
            return False
        root = root.right
    return True


def is_left_chain(root: Optional[TreeNode]) -> bool:
    while root:
        if root.right is not None:
            return False
        root = root.left
    return True


def draw(root: Optional[TreeNode], prefix: str = "", is_left: bool = True, is_root: bool = True) -> str:
    """Desen text al arborelui (dreapta sus, stânga jos)."""
    if root is None:
        return "(gol)\n" if is_root else ""
    out = ""
    if root.right:
        out += draw(root.right, prefix + ("    " if is_root else ("│   " if is_left else "    ")), False, False)
    out += prefix + ("" if is_root else ("└── " if is_left else "┌── ")) + str(root.val) + "\n"
    if root.left:
        out += draw(root.left, prefix + ("    " if is_root else ("    " if is_left else "│   ")), True, False)
    return out


def insertion_path(root: Optional[TreeNode], val: int) -> str:
    """Drumul de comparații pentru inserarea lui val (fără a insera)."""
    if root is None:
        return f"{val}: arbore gol -> devine rădăcină"
    steps = []
    cur = root
    while cur:
        if val < cur.val:
            steps.append(f"{val}<{cur.val} stânga")
            if cur.left is None:
                return f"{val}: " + ", ".join(steps) + f" -> copil stâng al lui {cur.val}"
            cur = cur.left
        elif val > cur.val:
            steps.append(f"{val}>{cur.val} dreapta")
            if cur.right is None:
                return f"{val}: " + ", ".join(steps) + f" -> copil drept al lui {cur.val}"
            cur = cur.right
        else:
            return f"{val}: " + ", ".join(steps + [f"{val}=={cur.val}"]) + " -> duplicat, neschimbat"
    return ""
```

```python
def inorder_trace(root: Optional[TreeNode]) -> List[str]:
    """Înregistrează când începe fiecare apel, când adaugă valoarea și când revine."""
    log: List[str] = []
    result: List[int] = []

    def dfs(node, depth):
        ind = "  " * depth
        if node is None:
            log.append(f"{ind}call(None) -> return")
            return
        log.append(f"{ind}start {node.val}")
        dfs(node.left, depth + 1)
        result.append(node.val)
        log.append(f"{ind}add {node.val}   result={result}")
        dfs(node.right, depth + 1)
        log.append(f"{ind}return {node.val}")

    dfs(root, 0)
    return log


def level_order_trace(root: Optional[TreeNode]) -> List[str]:
    """Coada după fiecare scoatere + adăugarea copiilor."""
    log: List[str] = []
    if root is None:
        return ["coadă goală -> []"]
    queue = deque([root])
    while queue:
        size = len(queue)
        log.append(f"Început nivel: coada={[n.val for n in queue]}, de scos={size}")
        level = []
        for _ in range(size):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
            log.append(f"  scos {node.val} -> coada={[n.val for n in queue]}")
        log.append(f"  nivel={level}")
    return log


def lecture_non_bst() -> TreeNode:
    """    2
          / \\
         1   3
        /
       4        (arbore binar care NU este BST)"""
    return TreeNode(2, TreeNode(1, TreeNode(4)), TreeNode(3))
```

Anexa C. Testele locale (`test_lab6.py`)

```python
"""Teste locale pentru Lab 6. Rulare:  python test_lab6.py"""

from lab6 import (TreeNode, insert_into_bst, insert_into_bst_recursive, build_bst,
                  inorder_traversal, level_order, height, snapshot, is_right_chain,
                  is_left_chain, draw, insertion_path, inorder_trace,
                  level_order_trace, lecture_non_bst)

BASE = [4, 2, 7, 1, 3, 6, 9]
passed = 0


def check(name, got, expected):
    global passed
    assert got == expected, f"FAIL {name}: got {got}, expected {expected}"
    passed += 1
    print(f"OK   {name}: {got}")


def section(title):
    print("\n" + "=" * 70 + f"\n{title}\n" + "=" * 70)


# ---------------------------------------------------------------- Problema 1
section("PROBLEMA 1 - construirea BST, pas cu pas")
root = None
for v in BASE:
    print(insertion_path(root, v))
    root = insert_into_bst(root, v)          # salvăm rădăcina returnată
    print(draw(root))

check("structura finală pe niveluri", level_order(root), [[4], [2, 7], [1, 3, 6, 9]])
check("valorile copiilor rădăcinii", (root.left.val, root.right.val), (2, 7))
check("înălțimea arborelui (muchii)", height(root), 2)

section("Predicție pentru 5 (pe un arbore nou)")
```

```
        fresh = build_bst(BASE)
        print(insertion_path(fresh, 5))
        fresh = insert_into_bst(fresh, 5)
        check("5 este copilul stâng al lui 6", fresh.right.left.left.val, 5)
        print(draw(fresh))

        section("Teste de margine - inserare")
        check("intrare goală", build_bst([]), None)
        one = build_bst([10])
        check("o valoare", (one.val, one.left, one.right), (10, None, None))
        neg = build_bst([0, -5, 5, -10, -1])
        check("valori negative (inorder)", inorder_traversal(neg), [-10, -5, -1, 0, 5])
        check("valori negative (niveluri)", level_order(neg), [[0], [-5, 5], [-10, -1]])
        dup = build_bst(BASE)
        before = snapshot(dup)
        dup = insert_into_bst(dup, 3)
        check("duplicat -> arbore neschimbat", snapshot(dup) == before, True)
        check("duplicate în listă", inorder_traversal(build_bst([5, 3, 5, 3, 8, 8])), [3, 5, 8])
        chain = build_bst([1, 2, 3, 4])
        check("[1,2,3,4] -> lanț doar spre dreapta", is_right_chain(chain), True)
        check("[1,2,3,4] -> înălțime 3", height(chain), 3)
        print(draw(chain))
        check("intrare sortată descrescător -> lanț stâng", is_left_chain(build_bst([4, 3, 2, 1])), True)

        # nodurile existente rămân pe loc
        t = build_bst(BASE)
        ids = (id(t), id(t.left), id(t.right))
        t2 = insert_into_bst(t, 8)
        check("nodurile existente păstrează poziția", (id(t2), id(t2.left), id(t2.right)) == ids, True)
        check("8 devine copilul stâng al lui 9", t2.right.right.left.val, 8)

        # varianta recursivă dă același arbore
        r = None
        for v in BASE:
            r = insert_into_bst_recursive(r, v)
        check("varianta recursivă = iterativă", level_order(r), level_order(build_bst(BASE)))

        # lanț lung: inserarea iterativă nu depășește stiva
        long_chain = build_bst(list(range(5000)))
        check("lanț de 5000 noduri (iterativ, fără recursie)", height(long_chain), 4999)

        # ---------------------------------------------------------------- Problema 2
        section("PROBLEMA 2 - traseu DFS inorder pe arborele din Problema 1")
        tree = build_bst(BASE)
        print("\n".join(inorder_trace(tree)))
        snap = snapshot(tree)
        check("inorder pe BST", inorder_traversal(tree), [1, 2, 3, 4, 6, 7, 9])
        check("arbore neschimbat după DFS", snapshot(tree) == snap, True)

        section("Teste DFS")
        nb = lecture_non_bst()
        snap_nb = snapshot(nb)
        check("arbore care nu e BST", inorder_traversal(nb), [4, 1, 2, 3])
        check("arbore non-BST neschimbat", snapshot(nb) == snap_nb, True)
        check("arbore gol", inorder_traversal(None), [])
        check("un nod", inorder_traversal(TreeNode(7)), [7])
        left_chain = TreeNode(3, TreeNode(2, TreeNode(1)))
        check("lanț stâng", inorder_traversal(left_chain), [1, 2, 3])
        right_chain = TreeNode(1, None, TreeNode(2, None, TreeNode(3)))
        check("lanț drept", inorder_traversal(right_chain), [1, 2, 3])

        # ---------------------------------------------------------------- Problema 3
        section("PROBLEMA 3 - traseu BFS (coada după fiecare scoatere)")
        tree = build_bst(BASE)
        print("\n".join(level_order_trace(tree)))
        snap = snapshot(tree)
        check("BFS pe BST", level_order(tree), [[4], [2, 7], [1, 3, 6, 9]])
        check("arbore neschimbat după BFS", snapshot(tree) == snap, True)

        section("Teste BFS")
        check("arbore gol", level_order(None), [])
        check("un nod", level_order(TreeNode(7)), [[7]])
        check("lanț drept", level_order(build_bst([1, 2, 3, 4])), [[1], [2], [3], [4]])
        check("lanț stâng", level_order(left_chain), [[3], [2], [1]])
        nb = lecture_non_bst()
        snap_nb = snapshot(nb)
        check("arbore din Problema 2", level_order(nb), [[2], [1, 3], [4]])
        check("arbore non-BST neschimbat", snapshot(nb) == snap_nb, True)

        print(f"\nToate cele {passed} teste au trecut.")
```

Anexa D. Ieșirea completă a testelor

```
==================================================================
PROBLEMA 1 - construirea BST, pas cu pas
==================================================================
4: arbore gol -> devine rădăcină
4
```

```
2: 2<4 stânga -> copil stâng al lui 4
4
    └── 2

7: 7>4 dreapta -> copil drept al lui 4
    ┌── 7
4
    └── 2

1: 1<4 stânga, 1<2 stânga -> copil stâng al lui 2
    ┌── 7
4
    └── 2
        └── 1

3: 3<4 stânga, 3>2 dreapta -> copil drept al lui 2
    ┌── 7
4
    │   ┌── 3
    └── 2
        └── 1

6: 6>4 dreapta, 6<7 stânga -> copil stâng al lui 7
    ┌── 7
    │   └── 6
4
    │   ┌── 3
    └── 2
        └── 1

9: 9>4 dreapta, 9>7 dreapta -> copil drept al lui 7
        ┌── 9
    ┌── 7
    │   └── 6
4
    │   ┌── 3
    └── 2
        └── 1

OK   structura finală pe niveluri: [[4], [2, 7], [1, 3, 6, 9]]
OK   valorile copiilor rădăcinii: (2, 7)
OK   înălțimea arborelui (muchii): 2

======================================================================
Predicție pentru 5 (pe un arbore nou)
======================================================================
5: 5>4 dreapta, 5<7 stânga, 5<6 stânga -> copil stâng al lui 6
OK   5 este copilul stâng al lui 6: 5
        ┌── 9
    ┌── 7
    │   └── 6
    │       └── 5
4
    │   ┌── 3
    └── 2
        └── 1


======================================================================
Teste de margine - inserare
======================================================================
OK   intrare goală: None
OK   o valoare: (10, None, None)
OK   valori negative (inorder): [-10, -5, -1, 0, 5]
OK   valori negative (niveluri): [[0], [-5, 5], [-10, -1]]
OK   duplicat -> arbore neschimbat: True
OK   duplicate în listă: [3, 5, 8]
OK   [1,2,3,4] -> lanț doar spre dreapta: True
OK   [1,2,3,4] -> înălțime 3: 3
            ┌── 4
        ┌── 3
    ┌── 2
1

OK   intrare sortată descrescător -> lanț stâng: True
OK   nodurile existente păstrează poziția: True
OK   8 devine copilul stâng al lui 9: 8
OK   varianta recursivă = iterativă: [[4], [2, 7], [1, 3, 6, 9]]
OK   lanț de 5000 noduri (iterativ, fără recursie): 4999

======================================================================
PROBLEMA 2 - traseu DFS inorder pe arborele din Problema 1
======================================================================
start 4
  start 2
    start 1
      call(None) -> return
    add 1   result=[1]
      call(None) -> return
    return 1
  add 2   result=[1, 2]
    start 3
      call(None) -> return
```

```
    add 3   result=[1, 2, 3]
      call(None) -> return
    return 3
  return 2
add 4   result=[1, 2, 3, 4]
  start 7
    start 6
      call(None) -> return
    add 6   result=[1, 2, 3, 4, 6]
      call(None) -> return
    return 6
  add 7   result=[1, 2, 3, 4, 6, 7]
    start 9
      call(None) -> return
    add 9   result=[1, 2, 3, 4, 6, 7, 9]
      call(None) -> return
    return 9
  return 7
return 4
OK   inorder pe BST: [1, 2, 3, 4, 6, 7, 9]
OK   arbore neschimbat după DFS: True

======================================================================
Teste DFS
======================================================================
OK   arbore care nu e BST: [4, 1, 2, 3]
OK   arbore non-BST neschimbat: True
OK   arbore gol: []
OK   un nod: [7]
OK   lanț stâng: [1, 2, 3]
OK   lanț drept: [1, 2, 3]

======================================================================
PROBLEMA 3 - traseu BFS (coada după fiecare scoatere)
======================================================================
Început nivel: coada=[4], de scos=1
  scos 4 -> coada=[2, 7]
  nivel=[4]
Început nivel: coada=[2, 7], de scos=2
  scos 2 -> coada=[7, 1, 3]
  scos 7 -> coada=[1, 3, 6, 9]
  nivel=[2, 7]
Început nivel: coada=[1, 3, 6, 9], de scos=4
  scos 1 -> coada=[3, 6, 9]
  scos 3 -> coada=[6, 9]
  scos 6 -> coada=[9]
  scos 9 -> coada=[]
  nivel=[1, 3, 6, 9]
OK   BFS pe BST: [[4], [2, 7], [1, 3, 6, 9]]
OK   arbore neschimbat după BFS: True

======================================================================
Teste BFS
======================================================================
OK   arbore gol: []
OK   un nod: [[7]]
OK   lanț drept: [[1], [2], [3], [4]]
OK   lanț stâng: [[3], [2], [1]]
OK   arbore din Problema 2: [[2], [1, 3], [4]]
OK   arbore non-BST neschimbat: True

Toate cele 33 teste au trecut.
```